
Installation

Bibliothekshackathon: KI-Workflows ohne Programmieren

Langflow 1.12.3 · September 2026

Gefördert durch:



Das KI-Servicezentrum Berlin-Brandenburg wird vom Bundesministerium für Forschung, Technologie und Raumfahrt unter dem Förderkennzeichen 16IS22092 gefördert.

Diese Anleitung erklärt, wie ihr den Bibliothekshackathon auf einem Laptop einrichtet: von der Installation von Docker bis zum ersten Flow. Sie richtet sich an Teilnehmende, die ihren eigenen Laptop einrichten, und an die Orga, die mehrere Laptops vorbereitet. Programmierkenntnisse braucht ihr nicht.

Das Einrichten dauert etwa 20 bis 30 Minuten, das meiste davon ist Warten auf Downloads. Hat die Orga euren Laptop schon vorbereitet, geht direkt zu Schritt 5.

Inhalt

1. Was ihr braucht
2. Docker installieren
3. Die Hackathon-Dateien herunterladen
4. Einstellungen eintragen
5. Langflow starten
6. Prüfen, ob alles funktioniert
7. Optional: ein lokales KI-Modell
8. Für die Orga: viele Laptops vorbereiten
9. Beenden, aktualisieren und entfernen
10. Wenn etwas nicht klappt

1. Was ihr braucht

	Mindestens	Für ein lokales KI-Modell
Arbeitsspeicher (RAM)	8 GB	16 GB
Freier Speicherplatz	10 GB	15 GB
Grafik	nicht nötig	am besten ein Mac mit Apple-Chip oder ein Laptop mit NVIDIA-Grafikkarte
Betriebssystem	Windows 10/11, macOS oder Linux	ebenso

Außerdem braucht ihr:

- **Administratorrechte** auf dem Laptop, um Docker zu installieren.
- **Internetzugang.** Langflow wird einmal heruntergeladen (knapp 1 GB), und die Bibliotheks-Bausteine fragen während der Arbeit Crossref, OpenAlex, Unpaywall und lobid ab.
- **Zugangsdaten von der Orga** für den KI-Cluster: eine Adresse, einen Gruppen-Schlüssel und einen Modellnamen. Ohne sie könnt ihr nur ein lokales Modell nutzen (siehe Schritt 7).

2. Docker installieren

Langflow läuft in **Docker**. Docker liefert ein Programm zusammen mit allem, was es braucht. Sonst muss auf dem Laptop nichts installiert werden.

Windows

1. [Docker Desktop](#) herunterladen und das Installationsprogramm ausführen.
2. Fragt das Installationsprogramm nach **WSL 2**, die Option eingeschaltet lassen. Docker braucht sie unter Windows.
3. Den Laptop neu starten, falls das Installationsprogramm darum bittet.
4. Docker Desktop öffnen und warten, bis unten links **Engine running** steht.

macOS

1. [Docker Desktop](#) herunterladen. Dabei die Version für euren Chip wählen: **Apple Silicon** für M1 und neuer, **Intel** für ältere Macs. Nachsehen könnt ihr das unter → *Über diesen Mac*.
2. Die heruntergeladene Datei öffnen und Docker in den Ordner *Programme* ziehen.
3. Docker Desktop öffnen und warten, bis unten links **Engine running** steht.

Linux

Ihr könnt Docker Desktop oder Docker Engine nutzen.

- **Docker Desktop:** der [Anleitung für eure Distribution](#) folgen.
- **Docker Engine:** der [Anleitung für eure Distribution](#) folgen, einschließlich des Compose-Plugins. Danach eurem Benutzer erlauben, Docker ohne sudo zu nutzen, und euch einmal ab- und wieder anmelden:

```
sudo usermod -aG docker $USER
```

Prüfen, ob Docker funktioniert:

```
docker compose version
```

3. Die Hackathon-Dateien herunterladen

Ohne Git (empfohlen, wenn ihr Git nicht kennt):

1. Die [Projektseite auf GitHub](#) öffnen.
2. Auf den grünen Knopf **Code** und dann auf **Download ZIP** klicken.
3. Die ZIP-Datei entpacken, z. B. auf den Schreibtisch.

Mit Git:

```
git clone https://github.com/aihpi/demo-bibliothekshackaton.git
cd demo-bibliothekshackaton
```

4. Einstellungen eintragen

Alle Einstellungen stehen in einer Datei namens `.env` im Projektordner. Im Ordner liegt dafür eine Vorlage, `.env.example`.

Die `.env` müsst ihr nicht selbst anlegen. Beim ersten Start von Langflow (Schritt 5) kopiert das Startskript die Vorlage nach `.env`, öffnet sie im Texteditor und hält an. Tragt die Werte ein, speichert die Datei und startet erneut.

Wer es lieber von Hand macht: `.env.example` kopieren, die Kopie `.env` nennen und ausfüllen.

Einstellung	Was hineingehört
KI_STANDARD	Welches Modell die Beispiel-Flows standardmäßig nutzen: <code>cluster</code> oder <code>lokal</code> . Auf <code>cluster</code> lassen, außer die Orga sagt etwas anderes.
CLUSTER_BASE_URL	Adresse des KI-Clusters, von der Orga
CLUSTER_API_KEY	Der Schlüssel eurer Gruppe, von der Orga. Bitte nicht weitergeben.
CLUSTER_MODELL	Name des Modells auf dem Cluster, von der Orga
LOKAL_BASE_URL	Adresse des lokalen Modells. Nur ändern, wenn ihr Ollama in Docker nutzt (siehe Schritt 7).
LOKAL_MODELL	Name des lokalen Modells, standardmäßig <code>qwen3.5:4b</code>
LOKAL_KONTEXT	Wie viel Text das lokale Modell auf einmal liest. So lassen.
KONTAKT_EMAIL	Eine E-Mail-Adresse. Crossref antwortet dann schneller, und Unpaywall findet mehr freie Volltexte.
OPENALEX_API_KEY	Ein kostenloser Schlüssel von openalex.org . Ohne Schlüssel bremst OpenAlex die Suche, wenn viele sie nutzen, und alle Gruppen im selben WLAN teilen sich das Limit.

Versteckte Datei auf dem Mac: Dateien, deren Name mit einem Punkt beginnt, sind im Finder versteckt. Mit `⌘ + ⬆ + .` (Befehlstaste, Umschalttaste, Punkt) macht ihr sie sichtbar.

5. Langflow starten

Docker Desktop muss laufen (unter Linux der Docker-Dienst). Dann:

- **Windows:** Doppelklick auf `starten.bat`.
- **macOS:** Doppelklick auf `starten.command`. Wenn macOS die Datei nicht öffnen will, siehe Wenn etwas nicht klappt.
- **Linux:** im Terminal im Projektordner `./starten.command` ausführen.

Beim ersten Start wird Langflow heruntergeladen, das dauert einige Minuten. Das Skript wartet, bis Langflow bereit ist, und öffnet dann <http://localhost:7860> im Browser.

Statt der Startskripte könnt ihr auch ein Terminal im Projektordner nutzen:

```
docker compose up -d
```

Dann <http://localhost:7860> selbst öffnen, sobald Langflow bereit ist, meist nach einer Minute.

6. Prüfen, ob alles funktioniert

1. In Langflow das Projekt **Starter Project** öffnen. Dort liegen die Beispiel-Flows.
2. **00 Erste Schritte – Hallo KI** öffnen.
3. Oben rechts auf **Playground** klicken, eine Frage eintippen, z. B. *Was ist eine DOI?*, und mit Enter abschicken.

Erscheint nach ein paar Sekunden eine Antwort, ist alles eingerichtet. Kommt stattdessen eine Fehlermeldung, siehe Wenn etwas nicht klappt.

Weiter geht es mit der Anleitung für Teilnehmende.

7. Optional: ein lokales KI-Modell

Standardmäßig nutzen die Flows den KI-Cluster. Ein lokales Modell läuft ganz auf eurem Laptop, es verlassen also keine Daten den Laptop. Das ist z. B. bei unveröffentlichten Abschlussarbeiten wichtig. Lokale Modelle sind allerdings kleiner, langsamer und machen mehr Fehler.

Das lokale Modell kommt von [Ollama](#). Es gibt drei Wege:

Weg	Wann	LOKAL_BASE_URL in der .env
Ollama-App auf dem Laptop	macOS (nutzt die Grafikeinheit), Windows mit NVIDIA-Grafikkarte	<code>http://host.docker.internal:11434/v1</code> (Standard)
Ollama in Docker	Linux, oder wenn sonst nichts installiert werden soll	<code>http://ollama:11434/v1</code>
Ollama auf einem Rechner im Raum , für alle	wenige starke Rechner, viele schwache Laptops	<code>http://<ip-adresse>:11434/v1</code>

Die Ollama-App installieren

- **Windows:** `OllamaSetup.exe` von ollama.com/download herunterladen und ausführen. Ollama läuft danach im Hintergrund, mit einem Symbol im Infobereich der Taskleiste.
- **macOS:** die App von ollama.com/download herunterladen, die Datei öffnen und Ollama in den Ordner *Programme* ziehen. Einmal starten. Danach läuft Ollama im Hintergrund, mit einem Symbol in der Menüleiste.
- **Linux:** im Terminal ausführen. Das installiert Ollama und richtet es als Dienst ein, der automatisch startet:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Prüfen, ob Ollama läuft: <http://localhost:11434> im Browser öffnen. Dort sollte **Ollama is running** stehen. Oder im Terminal:

```
ollama --version
```

Linux mit Docker Engine: Ollama hört standardmäßig nur auf 127.0.0.1, und Langflow in Docker erreicht diese Adresse nicht. Entweder Ollama in Docker nutzen oder Ollama auf allen Adressen hören lassen: `sudo systemctl edit ollama` ausführen, die beiden Zeilen unten eintragen, dann `sudo systemctl restart ollama`. Mit Docker Desktop ist das nicht nötig.

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

Das erste Modell herunterladen

Ein Terminal öffnen (unter Windows: *PowerShell* oder *Eingabeaufforderung*) und ausführen:

```
ollama pull qwen3.5:4b
```

Das lädt etwa 3,4 GB herunter und dauert je nach Verbindung einige Minuten. Danach prüfen, ob das Modell da ist:

```
ollama list
```

Dann eine erste Frage stellen. Die erste Antwort kann etwas länger dauern, weil das Modell erst in den Arbeitsspeicher geladen wird:

```
ollama run qwen3.5:4b "Was ist eine DOI?"
```

Erscheint eine Antwort, funktioniert das Modell. Der Laptop kann jetzt auch ohne Internet antworten. Die Bibliotheks-Bausteine brauchen das Internet aber weiterhin, um die Datenbanken abzufragen.

Ollama in Docker statt der App

```
docker compose --profile lokal up -d
```

Das startet Ollama in Docker und lädt beim ersten Mal das Modell aus `LOKAL_MODELL` herunter (einige GB). In der `.env` `LOKAL_BASE_URL=http://ollama:11434/v1` eintragen. Auf dem Mac nutzt dieser Weg die Grafikeinheit nicht und ist deutlich langsamer als die App.

Ein weiteres Modell in das Ollama in Docker laden:

```
docker compose exec ollama ollama pull qwen3.5:9b
```

Ollama auf einem gemeinsamen Rechner

Ollama dort mit der Umgebungsvariable `OLLAMA_HOST=0.0.0.0` starten, damit andere Laptops es erreichen, und die IP-Adresse dieses Rechners in `LOKAL_BASE_URL` eintragen.

Welches Modell?

- `qwen3.5:4b` (3,4 GB) läuft auf fast jedem Laptop.
- `qwen3.5:9b` (6,6 GB) liefert deutlich bessere Ergebnisse, braucht aber 16 GB RAM.

Wer ein anderes Modell als `qwen3.5:4b` nutzt, trägt dessen Namen in `LOKAL_MODELL` ein.

Das lokale Modell in Langflow nutzen

- **Für alle Flows:** in der `.env` `KI_STANDARD=lokal` setzen und danach noch einmal `docker compose up -d` ausführen, damit Langflow die Einstellung übernimmt.
- **Für einen einzelnen Flow:** im Baustein *KI-Modell* die **Quelle** auf *Lokal (Ollama)* stellen.

8. Für die Orga: viele Laptops vorbereiten

Jeden Laptop mit den Schritten 2 bis 5 vorbereiten: Docker installieren, den Projektordner kopieren, die `.env` der Gruppe hineinlegen und Langflow einmal starten, damit der Download vor der Veranstaltung erledigt ist.

WLAN schonen: Laden viele Laptops Langflow gleichzeitig herunter, wird das Netz langsam. Das Image einmal herunterladen und per USB-Stick auf die Laptops kopieren:

```
docker pull langflowai/langflow:1.12.3
docker save langflowai/langflow:1.12.3 -o langflow-1.12.3.tar

# danach auf jedem Laptop:
docker load -i langflow-1.12.3.tar
```

Zugangsdaten, Probelauf und den Ablauf des Tages beschreibt der Leitfaden für die Orga.

9. Beenden, aktualisieren und entfernen

Beenden

In Docker Desktop den Container stoppen, dessen Name mit `demo-bibliothekshackaton` beginnt, oder im Projektordner ausführen:

```
docker compose down
```

Eure eigenen Flows bleiben erhalten.

Auf die neueste Version aktualisieren

Bevor ihr aktualisiert:

- **Selbst gebaute Flows exportieren**, sicherheitshalber: in Langflow das Menü des Flows öffnen und **Export** wählen.
- **Die Beispiel-Flows nicht direkt ändern.** Bei jedem Start ersetzt Langflow die Beispiel-Flows im *Starter Project* durch die Fassungen aus dem Projektordner. Änderungen direkt in einem Beispiel-Flow gehen verloren, mit oder ohne Update. Arbeitet stattdessen an einer Kopie: in der Übersicht das Menü des Flows öffnen und **Duplicate** wählen.

Mit Git: im Terminal im Projektordner ausführen:

```
docker compose down
git pull
docker compose up -d
```

Weil Langflow vorher gestoppt wird, startet es neu und lädt die neuen Bausteine und Beispiel-Flows. `git pull` lässt eure `.env` unverändert.

Verweigert `git pull` das Update, weil ihr selbst eine Datei des Projekts geändert habt (z. B. den Port in `docker-compose.yml`), legt eure Änderungen beiseite, aktualisiert und holt sie dann zurück:

```
git stash
git pull
git stash pop
```

Ohne Git:

1. Langflow beenden (siehe oben).
2. Die neue Version herunterladen und entpacken (Schritt 3). Wie der neue Ordner heißt, ist egal.
3. Die `.env` aus dem alten Ordner in den neuen kopieren.
4. Langflow aus dem neuen Ordner starten (Schritt 5). Eure eigenen Flows sind noch da. Wenn alles funktioniert, könnt ihr den alten Ordner löschen.

Alles entfernen

```
docker compose down -v
docker image rm langflowai/langflow:1.12.3
```

Achtung: `down -v` löscht alle Flows, die ihr selbst gebaut habt. Vorher exportieren.

Danach den Projektordner löschen und Docker Desktop deinstallieren, falls ihr es nicht mehr braucht.

10. Wenn etwas nicht klappt

Problem	Lösung
Das Startskript meldet, Docker laufe nicht	Docker Desktop öffnen, auf Engine running warten und erneut starten.
Docker Desktop startet unter Windows nicht	WSL 2 fehlt. Eine Eingabeaufforderung als Administrator öffnen, <code>wsl --install</code> ausführen und neu starten. Hilft das nicht, ist vielleicht die Virtualisierung im BIOS ausgeschaltet.
macOS öffnet <code>starten.command</code> nicht	Rechtsklick auf die Datei und Öffnen wählen. Klappt das nicht, unter <i>Systemeinstellungen</i> → <i>Datenschutz & Sicherheit</i> auf Dennoch öffnen klicken.
„Permission denied“ beim Start unter macOS oder Linux	Im Terminal im Projektordner <code>chmod +x starten.command</code> ausführen.
Linux: „permission denied“ beim Verbinden mit Docker	Euer Benutzer ist noch nicht in der Gruppe <code>docker</code> (siehe Schritt 2). Nach dem Hinzufügen einmal ab- und wieder anmelden.

Problem	Lösung
Der Browser meldet „Seite nicht erreichbar“	Langflow startet noch. Eine Minute warten und die Seite neu laden.
Port 7860 ist belegt	In <code>docker-compose.yml</code> die Zeile "7860:7860" z. B. in "7861:7860" ändern und http://localhost:7861 öffnen.
Der Download bricht ab oder ist sehr langsam	Ein Firmen-Proxy oder ein volles WLAN. Das Image per USB-Stick laden (Schritt 8).
„401“ oder „Authentication“ im Flow	CLUSTER_API_KEY stimmt nicht. Bei der Orga nachfragen.
„Kein Modell gewählt“	CLUSTER_MODELL in der <code>.env</code> setzen oder im Baustein <i>KI-Modell</i> ein Modell auswählen.
Lokales Modell: „connection refused“	Ollama läuft nicht oder ist nicht erreichbar. Siehe Schritt 7, auch den Hinweis zu Linux.
Änderungen an der <code>.env</code> wirken nicht	Noch einmal <code>docker compose up -d</code> ausführen.
Alles andere	Das Protokoll von Langflow mit <code>docker compose logs -f langflow</code> ansehen oder die Orga fragen.

Weitere Probleme, die bei der Arbeit mit den Flows auftreten können, stehen in der Anleitung für Teilnehmende.